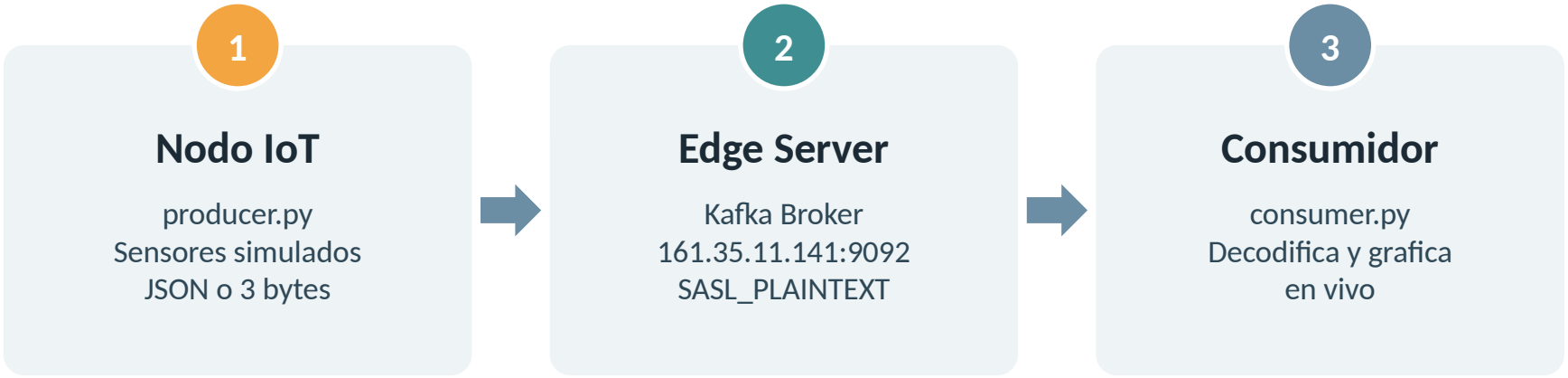
IoT: Estación Meteorológica

CC3067 Redes · Universidad del Valle de Guatemala

Javier Lopez — 23415

Apache Kafka · Python · topic 23415

Arquitectura de la solución



Patrón Publish/Subscribe: el productor publica en el topic 23415 con key "sensor1" cada 15–30 s; el consumidor se suscribe al mismo topic y actualiza la gráfica con cada mensaje. Ninguno conoce al otro: solo comparten el broker y el formato del mensaje.

3.1 Simulación de los sensores

Temperatura

Normal(μ=55, σ=15)
recortada a [0, 110] °C
float con 2 decimales

Humedad relativa

Normal(μ=55, σ=15)
recortada a [0, 100] %
entero

Dirección del viento

Uniforme sobre
{N, NO, O, SO, S, SE, E, NE}
8 categorías

Mensaje en JSON (63 bytes)

```
{"temperatura": 56.32, "humedad": 51, "direccion_viento": "SO"}
```

σ = 15 (varianza 225): ~99.7 % de las lecturas cae en [10, 100], así que casi no se recorta y la gráfica se centra en 55.

Preguntas 3.1 — JSON / SOAP

¿A qué capa OSI pertenece y por qué?

Capa 6, Presentación.

JSON y SOAP (XML) definen cómo se representan y serializan los datos: sintaxis, tipos y codificación de caracteres (UTF-8). Su trabajo es que emisor y receptor interpreten igual la información, sin importar el lenguaje o la plataforma de cada uno.

SOAP además se transporta sobre HTTP (aplicación). En el modelo TCP/IP todo esto cae en la capa de aplicación.

¿Qué beneficios tiene?

- Independiente del lenguaje: Python produce y cualquier cliente consume
- Autodescriptivo y legible: las claves dicen qué es cada valor
- Extensible: agregar un sensor no rompe a los consumidores
- Librerías de parseo y validación (JSON Schema, XSD) en todo lenguaje
- Desacopla el código del formato y lo hace menos frágil

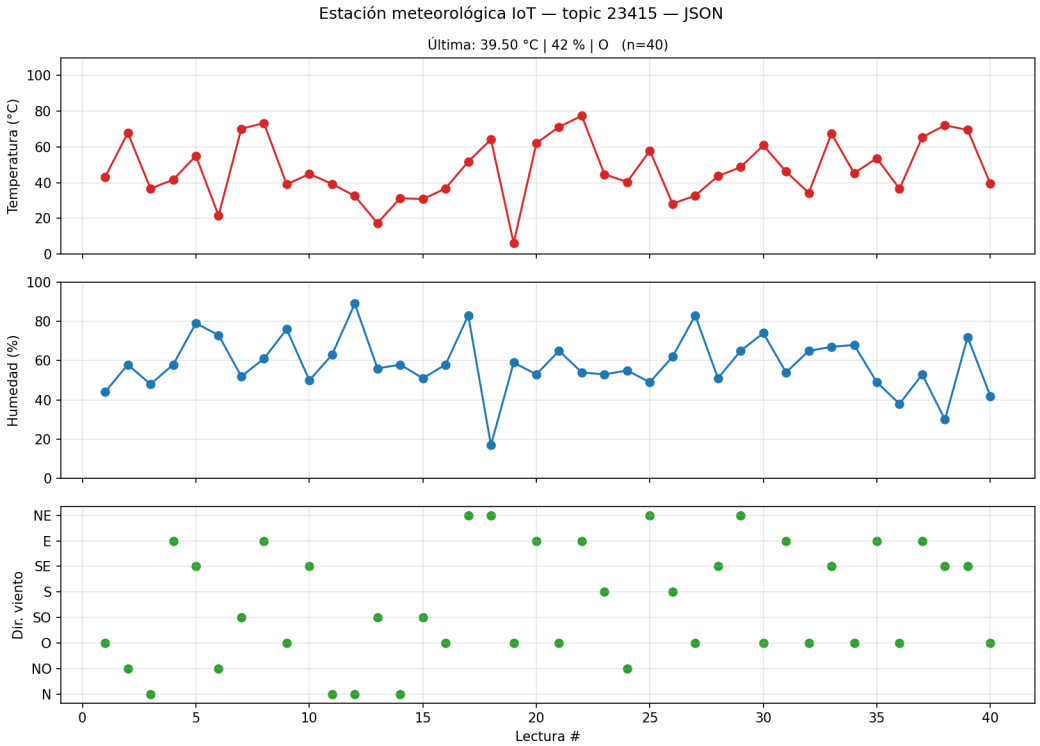
3.2 Envío de datos al Edge Server

```
conf = {
  "bootstrap.servers": "161.35.11.141:9092",
  "security.protocol": "SASL_PLAINTEXT",
  "sasl.mechanisms": "PLAIN",
  "sasl.username": "estudiante", ...}

p = Producer(conf)
while True:
  data = generar_data()
  payload = to_json(data)  # o encode()
  p.produce("23415", key="sensor1",
            value=payload)
  p.flush()
  time.sleep(uniform(15, 30))
```

- Cliente confluent-kafka (librdkafka) en Python
- Topic único: carné 23415, para no cruzar datos con otras parejas
- Key "sensor1": todos los mensajes del nodo van a la misma partición y se conserva el orden
- Intervalo aleatorio de 15–30 s; corre hasta Ctrl+C
- Un callback de entrega confirma la partición y el offset de cada mensaje

3.3 Consumo y gráfica en vivo (JSON)



- El consumidor se suscribe al topic 23415 con un group.id propio
- auto.offset.reset = earliest: grafica la telemetría pasada y la entrante
- La gráfica se redibuja con cada mensaje nuevo (matplotlib)
- 40 lecturas centradas en 55 °C / 55 %, viento repartido en las 8 direcciones

Preguntas 3.3 — Pub/Sub con Kafka

Ventajas

- Desacople entre productor y consumidor, en espacio y en tiempo
- Escala horizontal con particiones y grupos de consumidores
- Persistencia y replay por offsets
- Varios consumidores leen el mismo topic
- Replicación y tolerancia a fallos

Desventajas

- Infraestructura pesada de operar (brokers, KRaft/ZooKeeper)
- Cliente y TCP demasiado pesados para un microcontrolador
- Orden garantizado solo dentro de una partición
- Entrega at-least-once: puede haber duplicados
- SASL_PLAINTEXT manda las credenciales sin cifrar

¿Cuándo usar Kafka?

Sí: streaming de alto volumen, telemetría agregada en el edge/core, logs, event sourcing, pipelines en tiempo real, microservicios.

No: pocos mensajes, request/response síncrono, colas de tareas simples (RabbitMQ), sensores directos con pocos recursos (MQTT/CoAP), tiempo real duro.

3.4 Payload de 3 bytes (24 bits)



0 – 11000 < 2^{14} = 16384

0 – 100 < 2^{7} = 128

índice 0 – 7

Ejemplo: 56.32 °C · 51 % · SO

```
5632 → 01011000000000 | 51 → 0110011 | SO=3 → 011
packed = (temp << 10) | (hum << 3) | viento
bytes  = 58 01 9B  →  b'X\x01\x9b'  (3 bytes vs 63 en JSON)
decode: temp = (p >> 10) / 100, hum = (p >> 3) & 0x7F, viento = p & 0x7
```

3.4 Envío, consumo y gráfica con 3 bytes

[Insertar captura: gráfica con payload de 3 bytes]

[Insertar captura: terminal del productor]

[Insertar captura: terminal del consumidor]

Preguntas 3.4 — payload restringido

¿Qué complejidades introduce?

- Hay que diseñar un formato binario propio: se pierde la legibilidad y la autodescripción
- Acoplamiento fuerte: productor y consumidor deben compartir el layout exacto; versionarlo es difícil
- Cuantización: rango y precisión quedan fijos
- No caben metadatos: id, timestamp, checksum, versión
- Depurar es más difícil y hay que cuidar el endianness

¿Cómo cabe la temperatura en 14 bits?

Con punto fijo: se multiplica por 100 y se envía como entero.

El máximo es $110.00 \times 100 = 11000$, y $2^{14} = 16384 > 11000$, así que cualquier valor de [0, 110.00] con 2 decimales cabe en 14 bits.

Al decodificar se divide entre 100. No se pierde nada, porque el sensor ya tenía resolución de 0.01 °C.

Preguntas 3.4 — humedad float y Kafka

¿Y si la humedad es float con 1 decimal?

0.0–100.0 × 10 = 0–1000 → 10 bits. Total: 14 + 10 + 3 = 27 bits > 24.

Hay que ceder algo:

- Temperatura a 1 decimal: 0–1100 → 11 bits; 11 + 10 + 3 = 24 ✓
- Cuantizar la humedad a pasos de 0.5 o acotar rangos
- Subir el payload a 4 bytes, si la red lo permite
- Alternar sensores entre mensajes

Kafka con restricciones más fuertes

- compression.type (zstd, lz4, gzip) junto con linger.ms y batch.size: muchas lecturas por envío
- El timestamp y la key del record llevan el tiempo y el id del sensor sin gastar payload
- acks=0/1 para menos round-trips; max.request.size y message.max.bytes
- Gateway LoRa → Kafka (Kafka Connect / MQTT proxy) para que el nodo no hable Kafka
- retention.ms y log compaction en el broker

Conclusiones

- Kafka desacopla el nodo IoT del análisis y permite graficar el histórico y los datos en vivo.
- Con codificación binaria en punto fijo, el mensaje pasó de 63 a 3 bytes sin perder información.
- Repositorio: github.com/jlope384/IoT-estacion-metereologica